DP1. Sun shadows decided per receiver from the light's side: proven LIT, DARK and EDGE verdicts with outer and inner boxes, a light-space depth bound, and shadow caches that change the cost but never the picture

Defensive publication · draft 2 · 23 October 2026 · Author: Kyle Kimberlin (AI assistants helped write it) · Licence: CC BY 4.0

Keywords: shadow ray; hard shadow; directional light; sun shadow; moon shadow; visibility cache; shadow cache; per-receiver visibility; receiver classification; lit, umbra and partial classes; potential occluder list; shaft culling; swept volume; separating axis test; oriented bounding box; inscribed box; inner box; conservative bound; floating-point error bound; light space; light-space binning; shadow map; depth bound; minimum depth map; shadow ray interval; shadow ray culling; cache invalidation; dirty regions; tagged cache; GPU compute; implicit surfaces; superquadrics; moving occluders; motion envelope; Minkowski sum; erosion; validity window; verification mode; cache transparency; virtual reality; real-time rendering.

Classification (CPC): G06T 15/60 (shadow generation); G06T 15/06 (ray-tracing); G06T 15/50 (lighting effects); G06T 15/40 (hidden part removal); G06T 15/005 (general purpose rendering architectures); G06T 2210/12 (bounding box); G06T 2210/21 (collision detection, intersection).

Related disclosures in this series: DP2 (certified caches on time laws; drivers and frames), DP3 (exact integer time), DP5 (the physel primitive and drawing without meshes).

1. Field

Computer graphics: deciding which surface points a directional light, such as the sun or the moon, reaches in scenes of many solid primitives, in real time on CPUs and GPUs, with answers identical to a shadow ray traced per pixel.

2. Problem and known methods

A renderer may trace one exact shadow ray toward the sun per shaded pixel. Such rays can take much of the shading time. The question "can this point see the sun?" is asked again for every pixel of every frame, yet for a resting surface its answer depends neither on the camera nor on the frame.

Known methods	What the method here adds
Shadow maps (Williams 1978), adaptive (Fernando et al. 2001), sample-distribution (Lauritzen et al. 2011), page-cached virtual maps (Epic Games): depth from the light on a grid	Answers equal to a ray-traced reference in every bit; no resolution or bias
Irregular z-buffers, alias-free shadow maps (Aila and Laine 2004; Johnson et al. 2005; Wyman et al. 2015): exact at the visible samples, rebuilt per view	Answers kept across frames and shared by both eyes, as they do not depend on the camera
Ray-traced shadows and hybrids (Story 2015; Boksansky et al. 2019; AMD's FidelityFX Classifier, which rejects pixels by the shadow	Classes proven per receiver from exact boxes, not sampled maps;

Known methods	What the method here adds
cascades' minimum and maximum depths and takes ray intervals from them)	no reuse of earlier pixels
Light buffer, shadow caches (Haines and Greenberg 1986; Pearce 1991; Pearce and Jevans 1991): candidate occluders per direction; the last occluder tried first	Proofs that a whole receiver is lit or dark
Shaft culling (Haines and Wallace 1991), **hierarchical radiosity** (Hanrahan et al. 1991), **discontinuity meshing, backprojection** (Lischinski et al. 1992; Drettakis and Fiume 1994): potential occluders of a source-receiver pair; visible, partly and fully occluded classes	Classes proven down to the floating-point start rule of a per-pixel reference ray; full shadow proven from an occluder's interior
Local illumination environments (Fernandez, Bala and Greenberg 2002): per octree cell of an offline ray tracer, the visible lights and their potential occluders	Receivers that are the scene's own surfaces; a GPU cache kept true while the world changes; a verify mode
Temporal coherence (Glassner 1988; Jevans 1992); **visibility caching** with a set error (Popov et al. 2013); **occluder shrinking** for culling (Teller and Séquin 1991; Wonka et al. 2000; Décoret et al. 2003; Cohen-Or et al. 2003); **precomputed static shadows** (Sintorn et al. 2014); **kinetic certificates** (Basch et al. 1997)	Exact answers per receiver, with no fixed viewpoint, in changing scenes, with boxes grown and shrunk by proven amounts

The method aims at five properties at once: **exact** (every answer equals the per-pixel reference ray, bit for bit); **per receiver and camera-free** (kept across frames, shared by both eyes); **full shadow proven** from an occluder's interior; **checkable** (a verify mode compares every answer with the reference); **kept true while things change**.

3. Summary

Each surface that can receive the light (a sun-facing face of a primitive, a region of a rounded face, a whole small or carved primitive, a coarse stand-in, a ground tile) is classified once, from the light's side, against exactly the occluders the reference shadow ray can meet from it: **LIT** when no occluder's outer box, grown by a floating-point margin, meets the shaft its ray start points sweep toward the light; **DARK** when every start point's ray crosses one opaque occluder's inner box (proven inside its solid) at one shared depth; **EDGE** with at most six occluders, which the pixel alone tests; **OVERFLOW**, split into patches; **UNKNOWN**, which walks the reference ray. Verdicts are made off the frame (a CPU store while the world rests, a tagged GPU cache while it changes) and stood aside through light-space bins where something changed; a light-space relief map (Dtop, the nearest occluder-box depth; Dfull, the far side of an opaque inner box covering the texel) settles or shortens the remaining rays. A verify mode checks every answer against the reference in the same kernel: a cache changes the cost, never the picture.

4. Detailed description

4.1 Primitives, the light frame and the reference shadow ray

Primitives. The scene is made of small solid primitives (such as physels, DP5; superquadrics after Barr 1981), each with an oriented box: centre $\mathbf{c}$, unit axes $\mathbf{a}_0$, $\mathbf{a}_1$, $\mathbf{a}_2$ (from a quaternion), half-sizes $\mathbf{h}$. The method needs four facts from each kind of solid: (1) the solid lies inside its oriented box, the outer box (used by LIT, EDGE, Dtop); (2) zero to three inner boxes in closed form, each proven inside the solid (DARK, Dfull); (3) the exact ray cast never gives up, for example at an iteration cap, on a ray through an inner box (DARK, Dfull); (4) whether a pixel's stored surface point can lie slightly inside its own solid, as quadric roots can in single precision for grazing rays (then the receiver lists itself as an occluder). Inner boxes (half-sizes in the primitive's axes; taper $t \le 0.95$; bevel radius r), each scaled by $(1 - 10^{-6})$ and dropped if a half-size is ≤ 0:

Kind	Inner box or boxes
Box; a coarse stand-in box	**h**
Tapered box (frustum)	$(h_x(1 - t), h_y, h_z(1 - t))$
Ellipsoid	**h** / √3
Cylinder, axis y	$(h_x/\sqrt{2}, h_y, h_z/\sqrt{2})$
Cone (tapered cylinder)	$(h_x(1 - t)/\sqrt{2}, h_y, h_z(1 - t)/\sqrt{2})$
Bevelled box (the box shrunk by r, grown by a ball of radius r)	Three slabs: $(h_x, h_y - r, h_z - r)$, $(h_x - r, h_y, h_z - r)$, $(h_x - r, h_y - r, h_z)$
Bevelled frustum, cylinder or cone	The unbevelled inner box shrunk by r on every side
Convex superquadric, exponents n_s (sides), n_e (ends)	$(h_x\lambda w, h_y\lambda, h_z\lambda w)$: a = 2^(1/n_s) (1 for flat sides), λ = 1 / (a^(n_e) + 1)^(1/n_e) (1/a for flat ends), w = 1 − t(1 + λ)/2
Carved (a hole cut by another primitive); a superquadric tapered until not convex	None: never DARK

The iterative kinds (bevelled shapes, general superquadrics) prove DARK only once a test shows fact (3) for them: many test rays, a large share of them starting inside the box, none of which may miss or hit late.

The light frame. **L** is the unit vector toward the light; **U** = normalize(**Y** × **L**) (**X** in place of **Y** when |L_y| ≥ 0.99); **V** = **L** × **U**. A point **p** has light-space coordinates u = **p**·**U**, v = **p**·**V** and depth s = −**p**·**L**, growing away from the light.

The reference walk. A shadow ray from start point **o** along **L** tests, from t = 0 to the far start t_f, the resting primitives listed in a uniform grid (bounding sphere, then exact cast), multiplying the light through see-through primitives (glass, ice) and stopping at the first opaque hit; beyond t_f, the boxes of a coarse far grid, to t_end = 10 km. Moving primitives sit on a separate list that every ray tests. t_f = 2e_far/θ (e_far the far grid's largest error, θ the sun's angular diameter), so the sun's own width hides the coarse boxes' error. The method mirrors whatever reference walk an engine uses, segment by segment.

Start points that never depend on the camera. A pixel at point **p** with geometric normal **n** starts at **o** = **p** + δ**n**: for a primitive or stand-in, δ = e + max(e, min(1 mm, h_max)) + 2^{-17}(max|**c**| + 2h_max), with its own cast tolerance e = min(1 mm, 0.01 h_min) + 4.8 × 10^{-8} m; for the ground plane at height y_g, δ = 2^{-17} max(|y_g|, 1 m); for particles and liquid surfaces (no verdict), δ = 1 mm + 2^{-17} max|**p**|. The 2^{-17} terms cover single-precision rounding; the 1 mm term covers steps between neighbouring faces of one imported surface, which would otherwise show as dark seams. δ depends only on the receiver, so both eyes of a headset trace the same rays from a surface, and an answer can be kept for it.

4.2 Receivers

Receiver	Which	Point set (hull corners)	Pad
Box face	Each face of a sharp box (or box stand-in) with **n**·**L** ≥ 1/1024	The face moved out by δ: 4	0.17 mm + 0.005δ
Face region	Each of the six regions of a rounded, bevelled or tapered primitive whose box face has **n**·**L** ≥ −0.5	The frustum from the centre to the box face, beyond λ of the way out: 8	δ + 0.17 mm

Receiver	Which	Point set (hull corners) (5 if $\lambda = 0$)	Pad
Whole primitive	Carved and non-convex primitives; stand-ins that are not boxes	The box's 8 corners	δ + 0.17 mm
Ground tile	A 4-level clipmap about the viewer: 512 × 512 tiles of 0.25, 0.5, 1 and 2 m	The tile at $y_g + \delta$: 4	0.1 mm
Patch	A quarter of an OVERFLOW box face or tile (split up to twice: 16 patches) or face region (once); if all four quarters overflow, the parent stays OVERFLOW	The quarter's corners	As its parent

The shell λ is the largest share such that λ times the box, about its centre, lies inside an inner box: the largest $\min_i(b_i/h_i)$ over the inner boxes **b**, times $(1 - 10^{-4})$, rounded down to 1/64. A convex solid's surface never enters its inner box, so every surface point under a box face lies in that frustum, and one verdict serves the whole curved region.

Own occluder. Box faces, and regions of planar and bevelled kinds, never list their own primitive (a pixel uses such a region only if its normal points out of the primitive, $\mathbf{n}\cdot(\mathbf{p} - \mathbf{c}) > 0$). Ellipsoids, cylinders, cones and general superquadrics list themselves, since their single-precision hits for grazing rays seen from far away can lie millimetres inside; so do whole receivers. Stand-ins start their rays in the far grid ($t_f = 0$), so they never meet their own members. A clipmap window starts on a multiple of 64 tiles and moves only when the viewer leaves its middle half; then only the ground is classified again, the old tiles serving meanwhile.

Read rule. A pixel reads its receiver's verdict only when $\mathbf{n}\cdot\mathbf{L} \geq 1/1024$ and its point lies where the verdict was made: within 0.1 mm of the receiver's box on each box axis; for a box face, its normal within $\cos^{-1}$ 0.99999 of the face's and its point within 0.1 mm of the face's plane; for a region, at least $\lambda h_i - 0.1$ mm out along the region's axis; for the ground, $|y - y_g| \leq 0.1$ mm, on the finest level whose window holds it. Otherwise it walks. The pads ($\sqrt{3} \times 0.1$ mm) cover these margins.

4.3 Occluders and light-space bins

A receiver's occluders ("casters") are what the reference walk can meet from it: the near grid's resting primitives for $t \in [0, t_f]$ and the far grid's boxes for $t \in [t_f, t_{end}]$; moving primitives never (§4.9). Per caster, in double precision on the CPU: an id (bit 31 marks a far box), the oriented box, an opaque flag, up to three inner boxes, the footprint ($\mathbf{c}\cdot\mathbf{U} \mp e_U \mp g$, $\mathbf{c}\cdot\mathbf{V} \mp e_V \mp g$) and the depth range $(s_0, s_1) = -\mathbf{c}\cdot\mathbf{L} \mp (e_L + g)$, where $e_X = \Sigma_i h_i|\mathbf{a}_i\cdot\mathbf{X}|$. Float slack: $g = 2 \times 10^{-6}$ m $+ 4 \times 10^{-6} \times (\max|\mathbf{c}| + \Sigma h_i)$, twice that on the GPU; between caster k and receiver r, $\text{slack}(k, r) = 2 \times 10^{-6} + 4 \times 10^{-6} \times (\text{scale}_k + \text{scale}_r + |\mathbf{c}_k - \mathbf{c}_r|)$, with scale the largest centre coordinate plus the size. Footprints are binned in a uniform 2D grid over light space (near casters and far boxes apart), as prefix-summed starts and an item array; the bin size is the median footprint width, at least half of √(area / casters), grown ×1.25 until there are at most 4 million bins.

4.4 Classifying one receiver part

Figure 2 shows the two tests.

1. Query the bins with the part's footprint grown by its pad and the largest slack. On the CPU one query per primitive serves all its parts; each part tests only those hits, and each patch only its parent's.
2. For each candidate caster k (skipping the receiver's own primitive unless it lists itself), with g_k = pad + slack(k, r) and segment $(w_0, w_1) = (0, t_f)$ for a near caster or (t_f, t_{end}) for a far box:
    1. Reject if the depth ranges or footprints are apart by more than g_k (the shaft spans depths $s - w_1$ to $s - w_0$).
    2. Meet if a start point lies inside the box grown by g_k (when $w_0 = 0$), or the ray from the

hull's centroid crosses the box within $[w_0, w_1]$.

3. Otherwise test separating axes between the grown box and the swept hull: the box's three axes, the hull's face normals, each hull edge × **L**, each box axis × each hull edge, each box axis × **L**. Along axis **x** the swept hull projects to $[\min_p \mathbf{p}\cdot\mathbf{x} + \min(w_0\mathbf{x}\cdot\mathbf{L}, w_1\mathbf{x}\cdot\mathbf{L}), \max_p \mathbf{p}\cdot\mathbf{x} + \max(w_0\mathbf{x}\cdot\mathbf{L}, w_1\mathbf{x}\cdot\mathbf{L})]$ and the box to $\mathbf{c}\cdot\mathbf{x} \pm (\Sigma_i h_i|\mathbf{a}_i\cdot\mathbf{x}| + g_k|\mathbf{x}|)$. A gap proves they never meet.
4. If they may meet and the caster is opaque, uncarved and has inner boxes, test DARK; DARK ends the part.
5. Otherwise add the caster to the hits.

3. No hits: LIT. More than six hits, or more than two see-through: OVERFLOW. Otherwise EDGE, sorted near casters first, then by far-side depth s_1, largest first (nearest the receiver, so a shadowed ray stops at its first cast), then by index.

The DARK test. Put each start point **p** in the caster's frame: $q_i = \mathbf{a}_i\cdot(\mathbf{p} - \mathbf{c})$, $d_i = \mathbf{a}_i\cdot\mathbf{L}$. For an inner box **b** shrunk by $m = g_k$, the ray $\mathbf{p} + t\mathbf{L}$ is inside slab i for t between $(-b_i - q_i)/d_i$ and $(b_i - q_i)/d_i$ (if $|d_i| < 10^{-12}$, the point itself must lie inside the slab). Intersecting over three axes and all points gives $W = [t_{in}, t_{out}]$; the part is DARK if $\max(t_{in}, lo) < \min(t_{out}, hi)$, with $(lo, hi) = (0, t_f)$ for a near caster and $(t_f, 0.999\, t_{end})$ for a far box. Each end of t_f is widened outward ($\times 0.999 - 1$ cm, $\times 1.001 + 1$ cm) for the GPU's single-precision arc cosine.

```
classify(part R):
    hits = []
    for k in bins.query(R.footprint grown by R.pad + max_slack):
        if k == R.own: continue
        g = R.pad + slack(k, R)
        (w0, w1) = k.far ? (tf, T_END) : (0, tf)
        if apart(k, R, w0, w1, g): continue                    # depths or footprints
        if separated(box k grown by g, hull of R swept along L over [w0, w1]):
continue
        if k.opaque and k.inner and dark_through(k, R, g, k.far ? (tf, 0.999*T_END) :
(0, tf)):
            return DARK
        hits.append(k)
    if hits is empty: return LIT
    if len(hits) > 6 or count_see_through(hits) > 2: return OVERFLOW
    return EDGE(sort(hits, key = (far, -s1, id)))

dark_through(k, R, m, lo, hi):
    for b in k.inner:
        e = b - m
        if any(e <= 0): continue
        t_in, t_out = -inf, +inf
        for p in R.points:
            q = k.axes · (p - k.c);  d = k.axes · L
            for i in 0..2:
                if |d[i]| < 1e-12:
                    if |q[i]| >= e[i]: skip to next b
                else:
                    t0, t1 = sorted((-e[i] - q[i]) / d[i], (e[i] - q[i]) / d[i])
                    t_in = max(t_in, t0);  t_out = min(t_out, t1)
        if max(t_in, lo) < min(t_out, hi): return true
    return false
```

Worked example (Figure 2b). **L** = (0, 0.8, 0.6); an opaque, axis-aligned cube caster at (0, 2, 1.5), half-size 1, its own inner box; a ground tile 0.25 m wide at y = 0 centred on the origin (pads ignored). A corner $(x_0, 0, z_0)$ has the ray $(x_0, 0.8t, z_0 + 0.6t)$. The y slab gives $t \in [1.25, 3.75]$ for every corner; the z slab $[0.625, 3.958]$ for $z_0 = +0.125$ and $[1.042, 4.375]$ for $z_0 = -0.125$; the x slab holds every corner ($|x_0| < 1$, $L_x = 0$). So $W = [1.25, 3.75]$: every ray from the tile is inside the cube between t = 1.25 and 3.75 m, and with $t_f > 1.25$ m the tile is DARK.

4.5 Why each verdict is exact

Verdict	The pixel	Why it equals the reference walk
LIT	Light factor 1 (times a separate water factor where water lies above)	Every solid lies in its box; the box grown by the slack holds it whatever the rounding of the GPU's copy; the hull grown by the pad holds every start point of every pixel allowed to read the verdict. No reference ray can meet a caster
DARK	Light factor 0	For fixed t, the points **p** with **p** + t**L** in a box form a convex set (the box moved by −t**L**). All hull corners lie in it at a shared $t^* \in W$, so every point of the hull does, and every start point within the pad (the box was shrunk by it). Every reference ray thus crosses an inner box of an opaque, uncarved solid before t_f, and the walk's cast finds it (third fact of §4.1)
EDGE	Tests the listed casters exactly as the walk does	The list holds every caster a reference ray from the receiver can meet. Two see-through factors give the same product in either order ($a \times b = b \times a$ in IEEE floating point), and the walk keeps the movers' factor apart, so the answer has the walk's bits
OVERFLOW, UNKNOWN	The reference walk, shortened by the relief bound (§4.10)	It is the reference

4.6 The CPU store

Figure 5 shows the words. Per primitive index (stand-ins after the primitives), two 32-bit words. Word 0, the state word: bits 2f and 2f+1 hold face f's state (f = axis + 3 × [side > 0]; f = 0 for a whole receiver): 0 unknown, 1 LIT, 2 DARK, 3 EDGE or patches; bits 12-17 an OVERFLOW flag per face; bits 18-23 the shell λ in 64ths; bits 24-26 the receiver type (1 box faces, 2 regions, 3 regions that list themselves, 4 whole). Word 1: where its list entries start. List entries follow for the EDGE faces in face order, so face f's entry is found by skipping those of the EDGE faces below it. An EDGE entry is a header (bits 0-2 the caster count; bits 8-15 its length in words, the header included) and the caster indices (bit 31: a far box). A patch entry is a header with count 7, a word with the four quarters' 2-bit states (bits 0-7) and OVERFLOW flags (bits 8-11), then each quarter's entry; the quarter is $[x \geq 0] + 2[y \geq 0]$ for the point's coordinates along the part's two edges from its centre. The ground has one word per tile of the four windows: state (bits 0-1), OVERFLOW (bit 2), list offset (bits 8-31; a list past 2^{24} words is written OVERFLOW).

Made off the frame. The frame's thread copies the boxes of up to 32,768 primitives a frame from the world; a classification thread then works on half the hardware threads at the lowest priority, in jobs of 1,024 primitives or one row of tiles, giving up as soon as its inputs change. Its words reach the GPU in slices of 8 MB a frame, and shading reads the store once all of it has arrived. Its key: the light direction's exact bits and spread, the world's version, the materials, the primitive capacity and the far start's inputs. It is made only while nothing moves, but read while the world changes, its dirty bits standing aside what changed (§4.9). Jobs join in index order and lists are sorted, so the words are the same every run.

4.7 The GPU tagged cache

The CPU store is made over many frames on large scenes and is not made while things move; a second store on the GPU caches the verdicts of the receivers the camera sees. It has S primitive slots (a power of two; 262,144 by default, 16 MB) of 16 words: word 0 the state word (bits 0-26 as the CPU's, bit 31 valid); word 1 the light cone's id (§4.8); word 2 the faces' list lengths (3 bits a face) and, in bits 18-31, the tag $i >> \log_2 S$; words 3-15 up to 13 caster indices for all faces. Primitive i uses slot i mod S. Ground slots, one per tile of the windows (4×512^2), have 9 words: state, OVERFLOW, list length (bits 3-5) and valid (bit 31); the tile (i, j), 16 bits each; the cone id; up to 6 casters. A bit per slot says it may be filled, so invalidation reads only those. The GPU makes no patches.

A look, in compute passes after visibility:

1. **Marks.** One thread per pixel (each eye's, in a headset) marks its receiver in a bit mask: priority 1 if it has no verdict for a live cone; priority 2 if its verdict is for an older live cone and must be remade before it expires.
2. **List.** Prefix sums over the masks list the marked receivers in index order (primitives, then tiles; priority 1 first), up to the budget: the same list every run, whatever the GPU's timing.
3. **Claims.** Each listed primitive writes (look stamp << 14) | (16,383 − rank) to its slot's claim word by atomic maximum; the lowest rank wins and is classified. A primitive whose slot holds another's live verdict, seen in the last 4 looks, walks instead of evicting it. Stamps have 18 bits; at their end the claims are cleared.
4. **Classify.** One 64-thread workgroup per part. The items of the query's bins (up to 64 bins; larger queries go bin by bin) are numbered through in shared memory and shared out one by one, so a crowded bin is split too; a caster in several bins is tested only in the first bin of the query that holds it. A DARK proof sets a shared flag that stops the group; hits gather by atomic add (32 kept; more than six is OVERFLOW whichever they are); one thread sorts them. A receiver whose shaft meets this frame's dirty bins (§4.9) waits for a later frame.
5. **Pack.** One thread per receiver writes its slot and its bit.

The budget. The GPU's timers, read two frames late, set how many receivers a look may list (64 to 16,384) so that list and classify take about 0.25 ms a frame (one possible choice of budget). Looks run every frame while the world, the light, the CPU store or the windows change, one frame in eight while only the camera moves, and never while nothing changes. No result depends on wave size or timing (prefix sums, integer atomics, sorted lists); points are taken relative to the receiver's centre so sums stay small. A pixel reads the CPU store where its dirty bit is clear, else the GPU slot if its tag matches and its cone is live, else it is UNKNOWN; both stores name the receiver with the same bits (18-26), so one face test serves both.

4.8 A moving light: cones

A still light's cone is its exact direction, compared bit for bit. A moving light's cones have radius θ (0.5° by default); when the light passes the newest cone's centre, a new cone is made 0.9θ ahead along its path, so receivers are classified for the next cone before the last expires. Cone ids (32 bits) are never reused; the frame names up to three live cones that hold its direction (an angle, from atan2 of the cross and dot products, of at most $\theta - 10^{-6}$ rad), and only their verdicts are read.

For a cone with centre $\mathbf{L}_0$, any ray of the cone passes within $\tan\theta \times \Delta s$ of the central ray, Δs being the depth between them. So each caster grows by $\tan\theta \times \max(s_r - s_0, 0)$ (s_r the part's deepest depth, s_0 the caster's nearest), each inner box shrinks by as much, and far boxes start at $t_f \cos\theta$; near casters prove DARK only before $t_f \cos\theta$ and far boxes only before $0.999\, t_{end} \cos\theta$, because a ray of the cone goes up to $1/\cos\theta$ further than its depth along $\mathbf{L}_0$. The caster bins, made in a frame whose direction may differ from $\mathbf{L}_0$, are queried with a footprint grown by the tangent of the cone's radius plus that angle, times how long the shaft can stay in the grid's box (per start point and axis, the distance to the box's far side over $|L_i|$ minus that tangent). The bins are made again when the light turns 2° from their frame.

4.9 Keeping verdicts true while things change

The light's direction is in the CPU store's key and the cone ids; changes of resting matter (shapes, carves, see-through) reach dirty bins; the casters are the grid's resting primitives, less those whose moving bit (made on the GPU each frame) is set; the far start's inputs are in both stores' keys. Verify mode (§4.12) would show anything missed.

Dirty bins. In a frame with changes, the boxes of resting primitives that changed, joined or left the casters, and of far boxes whose drawn copy differs from the one the caster bins were made with, go into a 256 × 256 grid over light space, each bin keeping the smallest depth reached. Depths are ordered integers (a float's bits b become $b \mid 2^{31}$ if positive, $\neg b$ if negative, sorting as the floats do), so integer atomic minimum works. A header keeps the marked bins' rectangle and the smallest depth overall. A verdict is stood aside when its start points' footprint, grown by the widest live cone's tangent × (its deepest depth − the smallest dirty depth), meets a bin no deeper than its deepest depth: something changed between it and the light. A footprint over 4,096 bins counts as dirty. CPU verdicts are stood aside by a dirty bit per primitive and per tile; GPU slots are cleared.

Movers. A moving primitive is never a caster in a verdict, and a moving receiver reads none. Each moving group's bounding sphere goes into a 128 × 128 light-space grid holding the smallest depth reached (coarser while the spheres would mark more than 131,072 bins). A pixel answered by LIT or EDGE walks the movers alone where its bin holds one nearer the light than its start, and multiplies their factor in. Things that always move but are not primitives (cloth, rope) put a motion envelope, a sphere holding every pose they can take this frame, into the same grid.

4.10 The relief map: a light-space depth bound

Figure 3 shows the two depths along a ray.

Placement. An n × n map (1,024² by default; 64² to 4,096²) over light space, anchored to the world: its texel size T is a power of two and its corner a whole number of texels. It covers the visible receivers' light-space rectangle (every 4th pixel's surface, read back every 4th frame), grown by a quarter on each side and clipped to the union of the casters' footprints, outside which nothing can block a ray; it moves only when the receivers leave it or shrink to a quarter.

Two values per texel, as ordered 32-bit integers built on the GPU by atomic minimum (the order of work never changes them): **Dtop**, the smallest near-side depth $s_0 - tol$ of any caster box whose footprint touches the texel (every primitive and stand-in box: a superset of what the walk meets); **Dfull**, the smallest far-side depth $s_1 + tol$ of the inner box (shrunk by 0.1%) of an opaque, uncarved near-grid primitive of a closed-form kind, among those whose inner footprint, shrunk by tol, holds the whole texel.

Footprint test. A box seen along **L** projects to a hexagon, the sum of its projected half axes $g_i = (h_i\mathbf{a}_i\cdot\mathbf{U}, h_i\mathbf{a}_i\cdot\mathbf{V})$: the meet of three slabs, one across each g_i, with normal $\mathbf{n}_i \perp g_i$ and half-width $w_i = \Sigma_j|\mathbf{n}_i\cdot g_j|$. A texel at offset **d** from the hexagon's centre, half-size T/2, misses it if some slab has $|\mathbf{n}_i\cdot\mathbf{d}| - (T/2)(|n_{ix}| + |n_{iy}|) > w_i + tol$, and lies inside if every slab has $|\mathbf{n}_i\cdot\mathbf{d}| + (T/2)(|n_{ix}| + |n_{iy}|) < w_i - tol$, with $tol = 10^{-3} + 4 \times 10^{-6}(|u_c| + |v_c| + |s_c| + \Sigma h_i)$ rounding every bound outward. One thread per primitive; a footprint over 1,024 texels goes to a 256-thread workgroup. A coarser map (a cell per 8 × 8 texels) holds the smallest depth of any occupied water or loose-grain block, so most pixels skip the separate water factor.

Query, with $eps = 10^{-3} + 10^{-5}(|u| + |v| + |s|)$:

```
relief(o, tf):
    (u, v, s) = (o·U, o·V, -o·L)
    if (u, v) lies outside every caster footprint by eps: return LIT
    t = texel(u, v)
    if s + eps < Dtop[t]: return LIT
    reach = s - Dtop[t] + near_cell + eps
    if Dfull[t] is set and s - eps > Dfull[t] and reach < tf: return DARK
    return WALK(near walk to min(tf, reach),
                far walk to min(T_END, s - Dtop[t] + far_cell + eps), none when reach
<= tf)
```

Why exact. Along the ray the depth falls at unit rate: $s(\mathbf{o} + t\mathbf{L}) = s - t$. Every caster box touching the column starts at Dtop or deeper, so any blocker lies at $t \le s - Dtop$; one more grid cell covers the walk listing a primitive in the cells its CPU copy overlapped. If $s - eps > Dfull$, the column passes wholly through an opaque inner box whose far side lies between **o** and the light, so the ray is blocked within reach, inside the near walk. Other rays walk as before, only shorter. The map is built after a change of what rays see, when the light moves or when the map moves. In the form above it serves only while nothing moves (a mover makes it stand aside everywhere); §4.14 extends it to moving casters.

4.11 How a pixel gets its sunlight

```
sun(p, n, receiver):
    if n·L <= 0: return 0
    (o, tf) = start(p, n, receiver)                          # §4.1
    q = relief(o, tf)
    if q is LIT or DARK: f = q
    else:
        v = verdict(p, n, receiver)                          # read rule of §4.2; stores
of §4.6-4.7
```

```
        if v is LIT, DARK or EDGE:
            f = (v is EDGE) ? cast_list(o, tf, v.list) : v
            if f != 0 and movers_near(o): f *= walk_movers(o, tf)
        else:
            f = walk(o, tf, q.near_end, q.far_end)
    if f != 0 and water_near(o): f *= water(o)
    return f
```

The list cast tests each listed near caster as the walk does (bounding sphere, then exact cast in [0, t_f]; a see-through caster cast again for its exit and its factor multiplied in; an opaque hit before t_f returns 0), then each listed far box as a box from t_f to t_end. It uses the walk's own sphere test and cast function, with the same tolerance and range, in the same kernel, so the same instructions give the same bits (verify mode checks it, §4.12). The same function serves mirror hits, glints, surfaces seen through glass, ice and water, and light-probe rays. Shading is split: one thread per pixel shades every opaque LIT or DARK pixel that is not mirror-smooth and has no mover or water above it, with no ray code; the rest are queued, by prefix sums over per-tile masks in tile order, to edge, mirror and see-through kernels.

4.12 Verify mode: a cache changes the cost, never the picture

The rule. Verdicts and bounds are memoised facts about unchanged inputs: they may change how much work a pixel does, never what it shows. Anything not proven is traced exactly in the same frame.

Verify mode is a build of the shading kernels, chosen by a setting, in which every sun ray the relief bound or a verdict answered is also walked the reference way, from the same start point, in the same kernel, and the answers are compared bit for bit. Counters per site (shaded surfaces, mirror hits, see-through) and per verdict record rays, shares, casters listed, and disagreements in any bit and beyond the last bits (zero against non-zero, or a relative difference over 10^{-6}); the first 16 disagreements are stored with the pixel, verdict, receiver, both answers, the start point and what the walk met first. The pixel shows the cached answer, so the picture checked is the cache's own. The reference walk stays in the renderer as the fallback and the judge. A seeded random script (explosions, light steps, camera cuts, carves moved, glass turned to stone and back, primitives removed and added) must show zero disagreements in any bit, and the same pictures with the caches on and off.

4.13 What runs where

Work	Where	When
Copy primitive boxes from the world	CPU, frame thread, 32,768 a frame	After an input changes
Classify every resting receiver (CPU store); upload it, 8 MB a frame	CPU, half the hardware threads, lowest priority	While nothing moves, after an input changes
Moving bits, dirty bins, far-box watch, invalidation; caster bins	GPU compute	Each frame with a change; caster bins when the casters change or the light turns 2°
Mover bins	CPU	Each frame something moves
Marks, list, claims, classify, pack	GPU compute, per view	Frames that need a look, under the budget
Relief map	GPU compute	After a change of what rays see, the light or the map's place
Relief query, verdict read, list casts, walks, verify	GPU shading kernels	Every frame, per pixel

4.14 Moving casters

Figure 6 shows the idea. In §4.9-4.10 the relief map stands aside while anything moves, and movers are never in a verdict. This extension keeps every still cache valid while casters move within known bounds, using motion described by laws with envelopes (DP2): for a moving caster and a window of time steps, A is the largest distance any point of its solid can move from its listed pose.

1. **Swaying casters join verdicts** with the outer box grown by A on every side and each inner box shrunk by A (dropped where a half-size reaches zero). Every pose's solid lies within A of the listed one, so inside the grown box. A point of the shrunk inner box has a ball of radius A inside the listed inner box; its pre-image under a motion that moves points by at most A lies in that ball, so the point is in the solid at every pose. LIT and DARK hold for every pose of the window; only a band about ±A wide at shadow edges becomes EDGE, where the pixel casts the caster at its pose in this step.
2. **Still caches over still matter only.** The relief map, the verdicts, the light probes' geometry and the distance field are built over still matter; everything else enters through its envelope (the mover bins) and a grid of moving frames walked by rays. No cache switches off because something moves.
3. **The relief map per light cone**, built once per cone: Dtop from footprints grown by the cone's reach (tan θ times the depth span, bounded by the time a ray stays in the grid), Dfull from inner footprints shrunk by it; a ray whose mover bin holds something nearer walks the movers alone. Each build is sliced over frames into a second buffer: with cones of radius 0.5° and the sun at its real rate, the next cone is known about 108 s ahead (0.9θ = 0.45° at 360° a day).
4. **LIT and DARK carried from cone to cone:** only the edge band moves. A caster of height H casts a shadow H / tan e long at sun elevation e, so its edge moves H / sin²e per radian of elevation: about 0.70 m per degree for a 10 m caster at 30°, 0.14-0.31 m a cone.
5. **Work by the world's time scale.** The receivers to classify per frame are the visible receivers that want a verdict per cone divided by the frames a cone lives (0.9θ over the sun's angular rate, times the frame rate). A world whose clock runs k times the real rate asks for k times as many a frame; past the rate at which the looks' budget (§4.7) can classify them, receivers stay UNKNOWN and walk, which changes the cost and never the picture.

Under a dense canopy, leaves thinner than 2A have no inner box once shrunk, so the ground beneath reads EDGE or OVERFLOW, not DARK, unless one caster stands for the whole canopy.

5. Variants

The method may also be built in the following ways.

Lights: the moon and other directional lights, each with its own frame, cones and stores, at two bits per light per receiver; point and spot lights (the shaft is the hull of the start points and the light's position; a perspective frame, six maps for a point light; depth as distance from the light); area lights and the sun's or moon's disc (the shaft swept toward every point of the light, a Minkowski sum; DARK, the umbra, when every ray to every point of the light crosses one inner box at a shared depth; EDGE listing the penumbra's casters, whose share of the light the pixel computes exactly, such as the disc less the union of the casters' outlines, or by fixed quadrature); emissive primitives as lights; moving lamps through motion envelopes.

Primitives and bounds: triangle meshes (outer boxes per cluster; inner boxes found offline inside closed meshes by voxelizing the interior or searching for large inscribed boxes; none for open or alpha-tested meshes); voxels and sparse voxel trees (a solid voxel or a run of them is its own inner box); signed distance fields (an inner box where the distance stays negative, checked with the field's Lipschitz bound); point clouds and Gaussian splats (outer boxes at an opacity cut-off); CSG, NURBS and subdivision surfaces (boxes from control hulls); height fields, spheres, capsules and convex polytopes (inner boxes by linear programming); spheres, k-DOPs or convex hulls as inner or outer bounds.

Tests, sizes, precision: GJK or support-function tests, or exact shape-against-shaft tests for more LIT; interval, affine, rational or fixed-point arithmetic for slack and proofs; lists of 2 to 16 casters; more than two see-through casters with one fixed multiplication order (such as by index) in both the list and the walk; deeper or other patch splits; half, single, double or fixed-point precision, the slack scaled to match.

Receivers: lightmap or texture texels, meshlets, triangles, vertices, surfels, terrain patches, facade panels or solar cells, split adaptively until each is LIT, DARK or a short EDGE.

Stores and invalidation: hash tables keyed by receiver; compressed verdicts; verdicts saved to disk for still worlds and streamed with them, or shared over a network (a server may make them for every client); several cones per receiver; **arc verdicts**, per receiver the LIT, EDGE and DARK intervals over the light's daily arc, widened ±0.2° for the sun's change of declination, about 8-12 bytes a receiver; validity windows instead of or beside dirty bins, each verdict carrying the earliest step at which a moving caster could enter its shaft (a certificate from motion envelopes and largest rates), checked again by an event at that step (DP2); hierarchical dirty bins, or receivers indexed in light space; GPU slots claimed by other priorities (pixels covered, screen area), or set-associative.

Relief map: a pyramid of minimum and maximum depths, cascades, or several opaque intervals per texel; built by hardware conservative rasterization of the hexagons, or on the CPU; perspective or cube maps for local lights; used without verdicts, or verdicts without it.

Hardware: CPU alone or GPU alone; an integrated or second GPU making verdicts while the main GPU draws; hardware ray-tracing units for list casts or the fallback walk; any compute API; consoles, phones, standalone headsets, render farms; verify mode on a sample of pixels or frames, or in tests only.

Uses outside games: film and animation (verdicts reused across frames and shots of still sets); CAD, product and architectural visualization; daylight and planning studies (sun hours and solar access per facade panel or ground tile over a day or a year; shadow studies for planning permission); solar engineering (shading of photovoltaic cells and modules by nearby objects and other rows; heliostat fields); street shade and urban heat maps; light on crops and in greenhouses; robots, vehicles and drones (sun glare, solar charging, exposure planning); spacecraft (self-shadowing of solar panels and radiators for each attitude); simulation and training; line of sight to a distant radio transmitter; occlusion of a distant sound source (DP4).

6. Worked example

The DARK test is worked through in §4.4 (Figure 2b).

A start point (§4.1). A sharp box primitive of half-size 0.5 m on each axis, centred at (100, 2, 50) m, has the cast tolerance $e = \min(1\text{ mm}, 0.01 \times 0.5\text{ m}) + 4.8 \times 10^{-8}\text{ m} = 1.000048$ mm and the offset $\delta = e + \max(e, \min(1\text{ mm}, 0.5\text{ m})) + 2^{-17} \times (100\text{ m} + 2 \times 0.5\text{ m}) = 1.000048 + 1.000048 + 0.770569$ mm $\approx$ 2.7707 mm. Every pixel on its faces starts its sun ray 2.7707 mm out along the face's normal, whichever eye or camera sees it.

A state word (§4.6, Figure 5a). An axis-aligned ellipsoid under $\mathbf{L} = (0, 0.8, 0.6)$ lists itself (type 3). Its box faces have $\mathbf{n}\cdot\mathbf{L} = 0$ (±x), ±0.8 (±y) and ±0.6 (±z), so the regions of faces −x (f = 0), +x (f = 3), +y (f = 4) and +z (f = 5) are receivers ($\mathbf{n}\cdot\mathbf{L} \geq -0.5$), and those of −y (f = 1) and −z (f = 2) are not. Its inner box is $\mathbf{h}/\sqrt{3}$ scaled by $1 - 10^{-6}$, so $\lambda = (1/\sqrt{3})(1 - 10^{-6})(1 - 10^{-4}) = 0.57729$, or 36.95 sixty-fourths, rounded down to 36/64 = 0.5625. Say face 0 is LIT, face 3 DARK, face 4 LIT, and face 5 EDGE with one near caster, index 1,207, and one far box, index 5:

Field	Bits	Value	Adds
Face 0, LIT	0-1	1	1
Faces 1 and 2, not receivers	2-5	0	0
Face 3, DARK	6-7	2	128
Face 4, LIT	8-9	1	256
Face 5, EDGE	10-11	3	3,072

Field	Bits	Value	Adds
OVERFLOW flags	12-17	0	0
Shell λ in 64ths	18-23	36	9,437,184
Receiver type	24-26	3	50,331,648
Word 0			**59,772,289 = 0x03900D81**

Face 5 is the only EDGE face, so its list entry is the first at word 1's offset: three words, the header 2 | (3 << 8) = 0x00000302 (two casters; three words, the header included), then 0x000004B7 (caster 1,207: near casters first) and 0x80000005 (far box 5, bit 31 set). A pixel on the +z region with $\mathbf{n}\cdot\mathbf{L} \geq 1/1024$, lying at least 0.5625 h_z − 0.1 mm out along z, reads state 3, finds the entry and casts only the two listed casters.

A depth as an ordered integer (§4.9, §4.10). The depth s = 12.5 m has the float bits 0x41480000; it is positive, so it becomes 0x41480000 | 2^{31} = 0xC1480000. The depth s = −3.0 m has the bits 0xC0400000; it is negative, so it becomes ¬0xC0400000 = 0x3FBFFFFF. As unsigned integers 0x3FBFFFFF < 0xC1480000, as −3.0 < 12.5, so an integer atomic minimum keeps the nearer depth in whatever order the threads write.

7. Figures

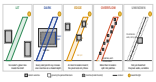


Figure 1. The five verdicts and their proofs, in side view (§3, §4.4).

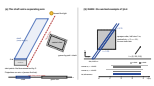


Figure 2. (a) a face's start points swept toward the light, and a caster's box grown by the pad and slack, with a separating axis; (b) the DARK test on the worked example of §4.4.

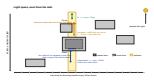


Figure 3. Dtop and Dfull along one texel's column, and the three answers a start point can get.



Figure 4. From receivers to pixels: the CPU store, the GPU cache, dirty bins, the pixel's order of questions and verify mode.

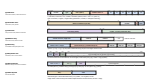


Figure 5. The state word, list and patch entries, a ground tile word, a GPU slot, a ground slot, a relief texel and a claim word.

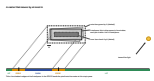


Figure 6. A caster moving by at most A, its grown outer box and shrunk inner box, and the ±A EDGE

band.

8. References

- Aila, T., and Laine, S. (2004). Alias-free shadow maps. *Eurographics Symposium on Rendering*, 161-166.
- AMD. FidelityFX Classifier 1.3 and FidelityFX Hybrid Shadows. FidelityFX SDK documentation, GPUOpen (read 22 October 2026).
- Barr, A. H. (1981). Superquadrics and angle-preserving transformations. *IEEE Computer Graphics and Applications*, 1(1), 11-23.
- Basch, J., Guibas, L. J., and Hershberger, J. (1997). Data structures for mobile data. *Proceedings of the 8th ACM-SIAM Symposium on Discrete Algorithms*, 747-756.
- Boksansky, J., Wimmer, M., and Bittner, J. (2019). Ray traced shadows: maintaining real-time frame rates. In E. Haines and T. Akenine-Möller (eds.), *Ray Tracing Gems*, Apress, 159-182.
- Cohen-Or, D., Chrysanthou, Y. L., Silva, C. T., and Durand, F. (2003). A survey of visibility for walkthrough applications. *IEEE Transactions on Visualization and Computer Graphics*, 9(3), 412-431.
- Décoret, X., Debunne, G., and Sillion, F. (2003). Erosion based visibility preprocessing. *Eurographics Symposium on Rendering*, 281-288.
- Drettakis, G., and Fiume, E. (1994). A fast shadow algorithm for area light sources using backprojection. *Proceedings of SIGGRAPH 94*.
- Epic Games. Virtual shadow maps in Unreal Engine. Unreal Engine documentation (read 22 October 2026).
- Fernandez, S., Bala, K., and Greenberg, D. P. (2002). Local illumination environments for direct lighting acceleration. *13th Eurographics Workshop on Rendering*, 7-14.
- Fernando, R., Fernandez, S., Bala, K., and Greenberg, D. P. (2001). Adaptive shadow maps. *Proceedings of SIGGRAPH 2001*, 387-390.
- Glassner, A. S. (1988). Spacetime ray tracing for animation. *IEEE Computer Graphics and Applications*, 8(2), 60-70.
- Haines, E. A., and Greenberg, D. P. (1986). The light buffer: a shadow-testing accelerator. *IEEE Computer Graphics and Applications*, 6(9), 6-16.
- Haines, E., and Wallace, J. (1991). Shaft culling for efficient ray-traced radiosity. *Second Eurographics Workshop on Rendering*; in *Photorealistic Rendering in Computer Graphics*, Springer (1994), 122-138.
- Hanrahan, P., Salzman, D., and Aupperle, L. (1991). A rapid hierarchical radiosity algorithm. *Computer Graphics (Proceedings of SIGGRAPH 91)*, 25(4).
- Jevans, D. A. (1992). Object space temporal coherence for ray tracing. *Proceedings of Graphics Interface '92*, 176-183.
- Johnson, G. S., Lee, J., Burns, C. A., and Mark, W. R. (2005). The irregular z-buffer: hardware acceleration for irregular data structures. *ACM Transactions on Graphics*, 24(4), 1462-1482.
- Lauritzen, A., Salvi, M., and Lefohn, A. (2011). Sample distribution shadow maps. *Symposium on Interactive 3D Graphics and Games (I3D)*.
- Lischinski, D., Tampieri, F., and Greenberg, D. P. (1992). Discontinuity meshing for accurate radiosity. *IEEE Computer Graphics and Applications*, 12(6), 25-39.
- Pearce, A. (1991). A recursive shadow voxel cache for ray tracing. In J. Arvo (ed.), *Graphics Gems II*, Academic Press, 273-274.
- Pearce, A., and Jevans, D. (1991). Exploiting shadow coherence in ray tracing. *Proceedings of Graphics Interface '91*.
- Popov, S., Georgiev, I., Slusallek, P., and Dachsbacher, C. (2013). Adaptive quantization visibility caching. *Computer Graphics Forum*, 32(2).
- Sintorn, E., Kämpe, V., Olsson, O., and Assarsson, U. (2014). Compact precomputed voxelized shadows. *ACM Transactions on Graphics*, 33(4).
- Story, J. (2015). Hybrid ray traced shadows. *Game Developers Conference 2015* (NVIDIA).
- Teller, S. J., and Séquin, C. H. (1991). Visibility preprocessing for interactive walkthroughs. *Computer Graphics (Proceedings of SIGGRAPH 91)*, 25(4).
- Williams, L. (1978). Casting curved shadows on curved surfaces. *Computer Graphics (Proceedings of SIGGRAPH 78)*, 12(3).
- Wonka, P., Wimmer, M., and Schmalstieg, D. (2000). Visibility preprocessing with occluder fusion for urban walkthroughs. *Eurographics Workshop on Rendering*.
- Wyman, C., Hoetzlein, R., and Lefohn, A. (2015). Frustum-traced raster shadows: revisiting irregular z-buffers. *Symposium on Interactive 3D Graphics and Games (I3D)*.

© 2026 Kyle Kimberlin. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/